Visualization Library Documentation

Objective

Create a comprehensive documentation guide for two popular Python visualization libraries: **Matplotlib** and **Pandas**. This guide focuses on the types of graphs each can generate, complete with practical code examples, descriptions, and a comparison section.

1. Library Overview

Matplotlib

Matplotlib is a foundational plotting library in Python. It allows users to create static, animated, and interactive visualizations with fine-grained control over every aspect of the figure.

Key Features:

- Extensive customization
- Wide range of plot types
- Great for scientific publications

Use Cases:

- Custom plots for academic papers
- Static visualizations

Pandas

Pandas is primarily a data manipulation library, but it includes built-in methods for quick visualizations using Matplotlib as a backend.

Key Features:

- Easy plotting from DataFrames
- Integrates seamlessly with data manipulation
- Quick EDA visualizations

Use Cases:

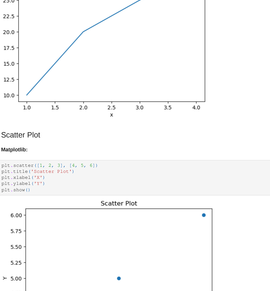
- Data inspection
- Simple and fast visualizations

In [1]:
```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
```

Line Plot

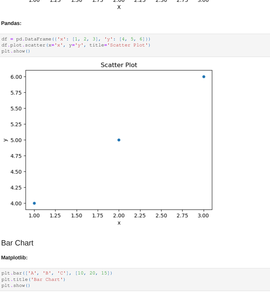
Matplotlib:

In [2]:
```
plt.plot([1, 2, 3, 4], [10, 20, 25, 30])
plt.title('Line Plot')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.show()
```



Pandas:

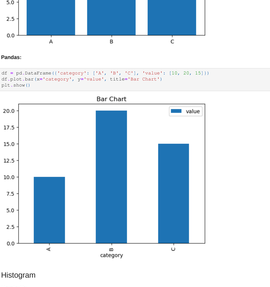
In [3]:
```
df = pd.DataFrame({'x': [1, 2, 3, 4], 'y': [10, 20, 25, 30]})
df.plot(x='x', y='y', title='Line Plot')
plt.show()
```



Scatter Plot

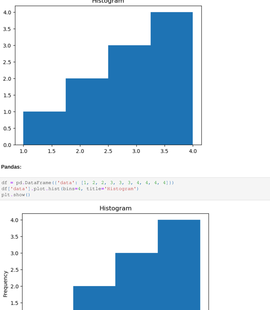
Matplotlib:

In [4]:
```
plt.scatter([1, 2, 3], [4, 5, 6])
plt.title('Scatter Plot')
plt.xlabel('X')
plt.ylabel('Y')
plt.show()
```



Pandas:

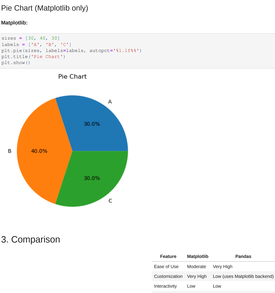
In [5]:
```
df = pd.DataFrame({'x': [1, 2, 3], 'y': [4, 5, 6]})
df.plot.scatter(x='x', y='y', title='Scatter Plot')
plt.show()
```



Bar Chart

Matplotlib:

In [6]:
```
plt.bar(['A', 'B', 'C'], [10, 20, 15])
plt.title('Bar Chart')
plt.show()
```



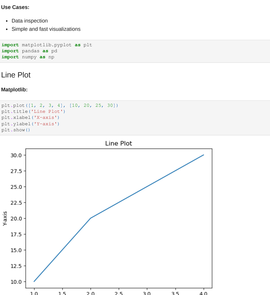
Pandas:

In [7]:
```
df = pd.DataFrame({'category': ['A', 'B', 'C'], 'value': [10, 20, 15]})
df.plot.bar(x='category', y='value', title='Bar Chart')
plt.show()
```

Histogram

Matplotlib:

In [8]:
```
values = [1, 2, 2, 3, 3, 3, 4, 4, 4, 4]
plt.hist(values, bins=3)
plt.title('Histogram')
plt.show()
```



Pandas:

In [9]:
```
df = pd.DataFrame({'data': [1, 2, 2, 3, 3, 3, 4, 4, 4, 4]})
df['data'].plot.hist(bins=4, title='Histogram')
plt.show()
```


Pie Chart (Matplotlib only)

Matplotlib:

In [10]:
```
sizes = [35, 40, 30]
labels = ['A', 'B', 'C']
plt.pie(sizes, labels=labels, autopct='%1.1f%%')
plt.title('Pie Chart')
plt.show()
```


3. Comparison

Feature	Matplotlib	Pandas
Ease of Use	Moderate	Very High
Customization	Very High	Low (uses Matplotlib backend)
Interactivity	Low	Low
Performance	High	High
Integration	Manual	Seamless with DataFrames
Plot Variety	Extensive	Limited

Conclusion

- **Matplotlib** is best suited for detailed, custom visualizations with precise control.
- **Pandas** is ideal for quick and simple plots directly from your DataFrame.

Use **Pandas** for fast EDA and **Matplotlib** when you need advanced visualizations.

In [ ]:
```
For reference
Matplotlib :https://matplotlib.org/stable/users/explain/quick_start.html#quick-start
Pandas :https://pandas.pydata.org/docs/user_guide/index.html
```